

软件安全作业三 —— 安全编程题

班级：2022211801 学号：2022211570 姓名：项枫

一、常见的格式化字符串操作错误

1、缓冲区溢出

缓冲区溢出是指在向字符数组写入数据时，如果没有正确检查目标缓冲区的大小，数据就会超出预定的缓冲区范围，导致内存覆盖，严重时会导致程序崩溃、数据损坏或者安全漏洞（例如缓冲区溢出攻击）。

2、格式化符号不匹配

格式化符号和提供的参数类型不匹配。`printf` 和 `sprintf` 等函数使用格式化符号来确定如何处理参数类型。类型不匹配会导致未定义行为，例如内存损坏或程序崩溃。

3、传递参数数量不一致

格式化字符串中的占位符数量与实际传递的参数数量不一致，可能导致堆栈损坏或者读取错误的内存。

4、缺少格式化符号的参数

格式化字符串中使用了占位符，但实际调用时没有传递足够的参数，可能导致访问未初始化或错误的内存。

5、使用未初始化的参数

格式化字符串操作时，如果使用了未初始化的参数，可能导致不可预测的行为。

二、格式化字符串错误的缓解措施

1、始终确保缓冲区大小足够

在进行字符串格式化时，要确保目标缓冲区足够大，避免发生缓冲区溢出。可以使用 `snprintf`、`vsnprintf` 等函数来限制写入的字符数。

2、验证格式化符号和参数类型一致性

在格式化字符串操作前，确保格式化符号与实际传递的参数类型匹配。例如，整数应使用 `%d`、`%i`，字符串应使用 `%s`，浮点数应使用 `%f` 等。

3、检查参数的数量和匹配

确保传递给格式化字符串的参数数量与格式化符号所需的参数数量一致，避免遗漏或者过多的参数。

4、使用更安全的函数

在处理字符串时，优先使用安全的字符串操作函数，例如 `snprintf`、`vsprintf`，避免使用不安全的 `sprintf`、`vsprintf` 等。这些安全函数允许你指定最大写入字符数，防止溢出。

5、避免未初始化变量

在使用变量之前，确保其已经正确初始化。使用调试器（如 GDB）或者静态分析工具来帮助识别未初始化的变量。

三、编写函数

1、函数编写思路 and 流程

函数首先检查传入的目标缓冲区指针和大小是否有效，若无效则返回 `-1` 表示参数无效。

使用 `va_list` 类型的变量 `args` 来处理可变数量的参数，通过 `va_start` 宏初始化。

使用一个循环遍历格式化字符串 `format`，逐个处理其中的字符：对于普通字符，直接将其复制到目标缓冲区中；对于 `%s` 格式符，从参数中获取字符串，计算其长度，然后将其复制到目标缓冲区中；对于 `%d` 格式符，从参数中获取整数，将其转换为字符串，然后复制到目标缓冲区中。

在复制字符串时，会检查缓冲区是否足够大，若不足则返回 `-1`。

循环结束后，添加字符串终止符 `\0` 到目标缓冲区的末尾。

最后，释放可变参数列表并返回写入目标缓冲区的字符数量（不包括终止符）。如果参数不匹配或缓冲区不足，也返回 `-1`。

2、函数源代码

```
#include <stdio.h>
#include <stdarg.h>
#include <string.h>

// 辅助函数：将字符串复制到缓冲区，并检查缓冲区是否足够大
int copy_to_buffer(char* buffer, size_t* index, size_t sizeOfBuffer, const char* str) {
    size_t len = strlen(str); // 获取字符串长度
    if (*index + len < sizeOfBuffer) {
        // 如果缓冲区足够大，将字符串复制到缓冲区
        memcpy(buffer + *index, str, len);
        *index += len; // 更新写入位置
        return 0;
    }
    return -1; // 缓冲区不足，返回错误
}

// 自定义 sprintf_s 函数，实现简单的格式化输出
int my_sprintf_s(char* buffer, size_t sizeOfBuffer, const char* format, ...) {
    if (buffer == NULL || sizeOfBuffer == 0) {
        // 参数无效，返回错误
        return -1;
    }

    va_list args;
    va_start(args, format); // 初始化 va_list
    size_t i = 0; // 记录写入字符的位置

    // 遍历格式化字符串
    while (*format != '\0') {
        if (*format != '%' && i < sizeOfBuffer - 1) {
            // 处理普通字符，直接复制到缓冲区
            buffer[i++] = *format;
        }
        else if (*format == '%' && *(format + 1) == 's') {
            // 处理 %s 格式符
            const char* str = va_arg(args, const char*); // 获取下一个参数（字符串）
            if (str == "-1") {
                // 如果字符串参数为"-1"，返回错误
                va_end(args);
                return -1;
            }
            if (str == NULL || copy_to_buffer(buffer, &i, sizeOfBuffer, str) != 0) {

```

```

        // 如果字符串为空或者缓冲区不足，返回错误
        va_end(args);
        return -1;
    }
    format++; // 跳过 '%s'
}
else if (*format == '%' && *(format + 1) == 'd') {
    // 处理 %d 格式符
    long int num = va_arg(args, long int); // 获取下一个参数 (整数)
    char str[200];
    sprintf_s(str, sizeof(str), "%d", num); // 将整数转换为字符串
    if (copy_to_buffer(buffer, &i, sizeofBuffer, str) != 0) {
        // 如果缓冲区不足，返回错误
        va_end(args);
        return -1;
    }
    format++; // 跳过 '%d'
}
else {
    // 处理未知的格式符，返回错误
    va_end(args);
    return -1;
}

++format; // 继续处理下一个字符
}

// 在字符串末尾添加终止符
if (i < sizeofBuffer) {
    buffer[i] = '\0';
} else {
    // 缓冲区不足，返回错误
    va_end(args);
    return -1;
}

va_end(args); // 清理 va_list
return i; // 返回写入的字符数 (不包括终止符)
}

int main() {
    char buffer[20];

    // 示例 1: 正常输出

```

```
printf("Normal output example:\n");  
int result = my_sprintf_s(buffer, sizeof(buffer), "BUPT, %s%d!", "SCSS, ", 1111, "-1");  
  
if (result >= 0) {  
    printf("Formatted string: %s\n", buffer);  
} else {  
    printf("Error occurred: %d\n", result);  
}  
  
// 示例 2：缓冲区溢出示例  
printf("Example of Scalable Buffer:\n");  
result = my_sprintf_s(buffer, sizeof(buffer),  
"BUUUUUUUUUUUUUUUUUUUUUUUUUUPPPPPPPPTTTTTTTTTTTT, %s%d!", "SCSS, ", 1111, "-1");  
  
if (result >= 0) {  
    printf("Formatted string: %s\n", buffer);  
} else {  
    printf("Error occurred: %d\n", result);  
}  
  
// 示例 3：查看堆栈内容  
printf("Example of viewing stack content:\n");  
result = my_sprintf_s(buffer, sizeof(buffer), "BUPT, %s%d%s%s!", "SCSS, ", 1111,  
"-1");  
  
if (result >= 0) {  
    printf("Formatted string: %s\n", buffer);  
} else {  
    printf("Error occurred: %d\n", result);  
}  
  
// 示例 4：内存覆盖示例  
printf("Memory Overwrite Example:\n");  
result = my_sprintf_s(buffer, sizeof(buffer), "BUPT, %s%d%s!", "SCSS, ", 1111,  
"Memory Overwrite Memory Overwrite Memory Overwrite", "-1");  
  
if (result >= 0) {  
    printf("Formatted string: %s\n", buffer);  
} else {  
    printf("Error occurred: %d\n", result);  
}  
  
return 0;  
}
```

3、运行结果

```
"D:\SoftwareSecurity\homew... x + v
Normal output example:
Formatted string: BUPT, SCSS, 1111!
Example of Scalable Buffer:
Error occurred: -1
Example of viewing stack content:
Error occurred: -1
Memory Overwrite Example:
Error occurred: -1

Process returned 0 (0x0)   execution time : 0.061 s
Press any key to continue.
```

test1：正确输出。

test2：验证可伸展的缓冲区和程序崩溃，报错中断。

test3：验证查看栈内容，报错中断。

test4：验证内容覆写，报错中断。